From Symmetric Pattern Matching to Quantum Control

Benoît Valiron
CentraleSupélec – LRI, U. Paris Saclay

Joint work with Amr Sabry (Indiana U.)
and Juliana Vizzotto (U. Federal de Santa Maria)

Universidad de Buenos Aires 2018

Roadmap

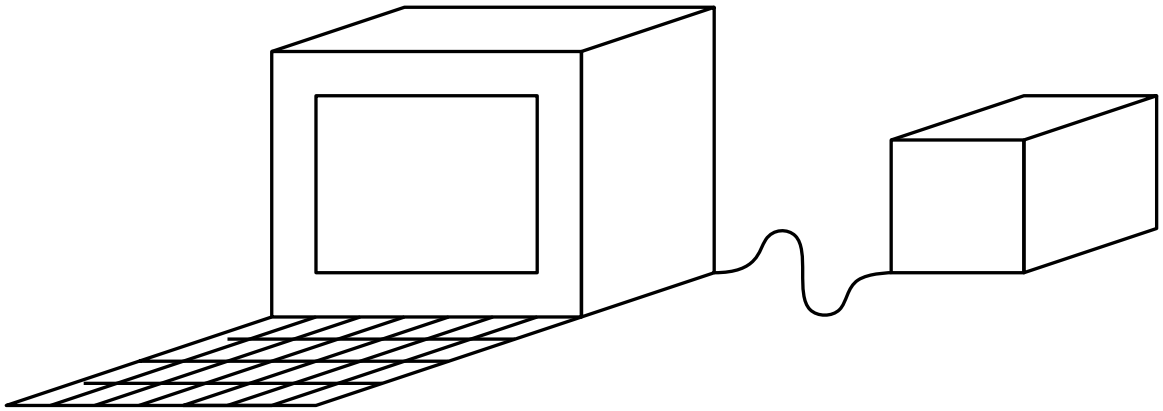
- Quantum and Control
  - Model of Computation
  - Notion of Control in Quantum Computation
- Previous Approaches
- Pattern Matching and Quantum Control

Roadmap

- Quantum and Control
  - Model of Computation
  - Notion of Control in Quantum Computation
- Previous Approaches
- Pattern Matching and Quantum Control

Model of Computation

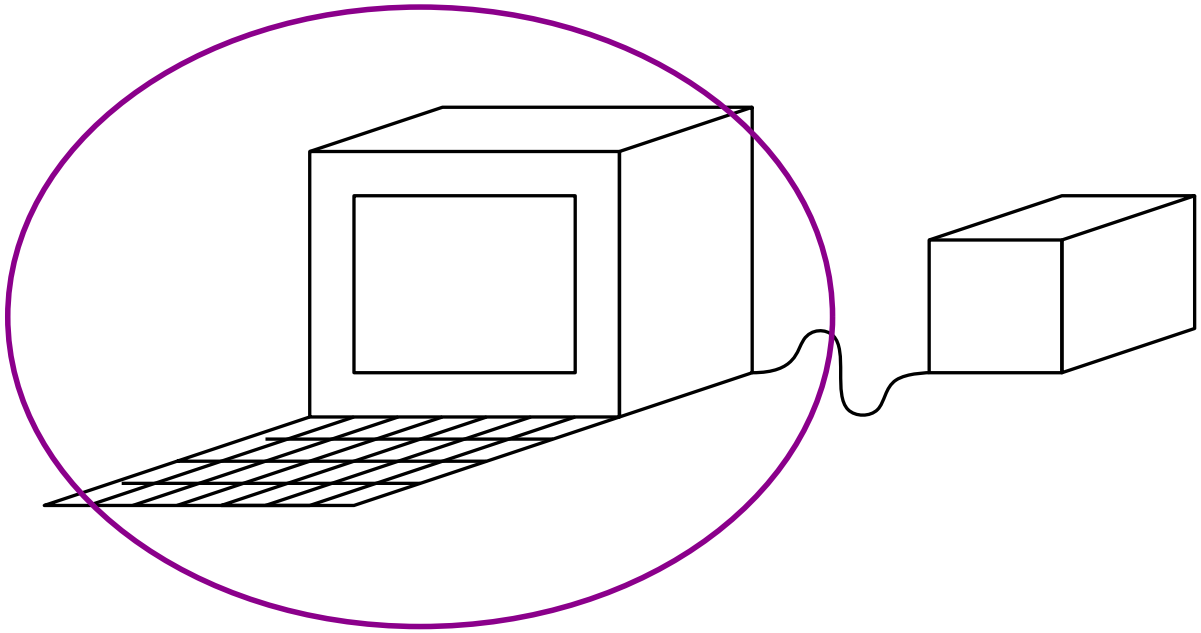
Quantum computation with classical control



A quantum co-processor

Model of Computation

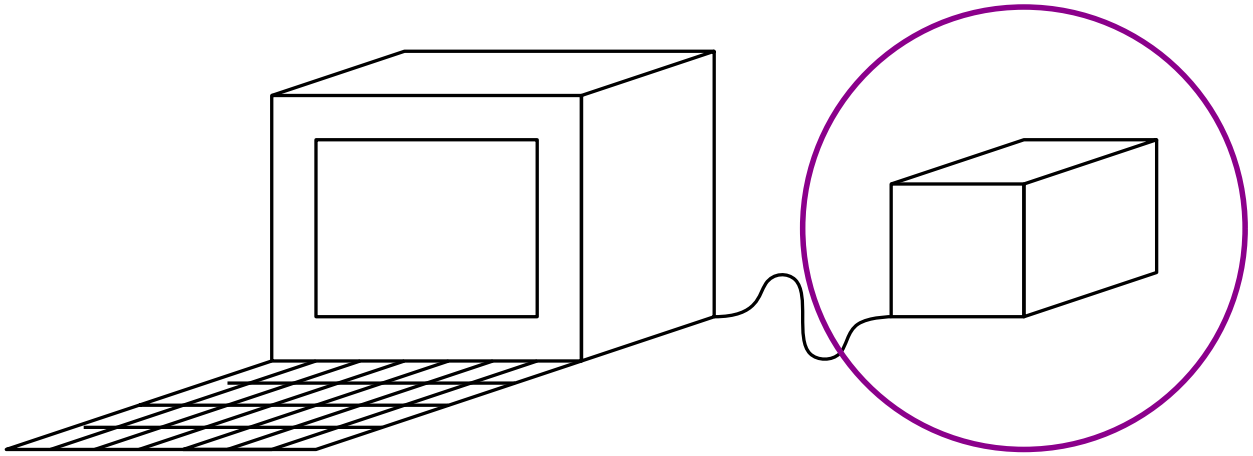
Quantum computation with classical control



The program lives here

Model of Computation

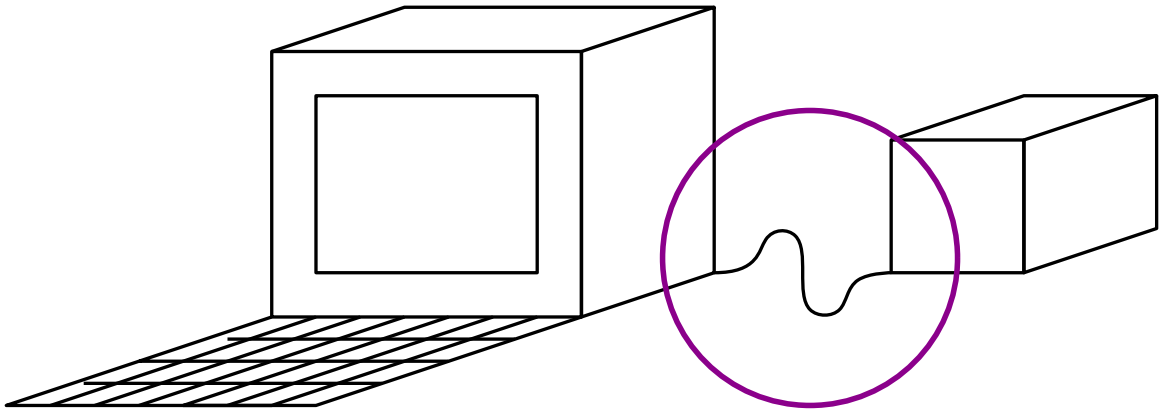
Quantum computation with classical control



This only holds quantum data

Model of Computation

Quantum computation with classical control



Series of instructions/circuit and feedback

Quantum Memory

A quantum memory with n quantum bits is a complex combination of strings of n bits in a Hilbert space. E.g. for $n = 3$:

$$\begin{array}{rr} & -\frac{1}{2} \cdot \mathsf{0\,0\,0} \\ + & \frac{1}{2} \cdot \mathsf{0\,0\,1} \\ + & \frac{i}{2} \cdot \mathsf{1\,1\,0} \\ - & \frac{i}{2} \cdot \mathsf{1\,1\,1} \end{array}$$

with a norm condition.

Unlike probabilistic distributions,

all are available at the same time.

Quantum Memory

A quantum memory with n quantum bits is a complex combination of strings of n bits in a Hilbert space. E.g. for $n = 3$:

$$
\begin{array}{rrl}
 & -\frac{1}{2} & \cdot\, \boxed{0}\,0\,0 \\
+ & \frac{1}{2} & \cdot\, \boxed{0}\,0\,1 \\
+ & \frac{i}{2} & \cdot\, \boxed{1}\,1\,0 \\
- & \frac{i}{2} & \cdot\, \boxed{1}\,1\,1 \\
 & & \text{qubit 1}
\end{array}
$$

with a norm condition.

Unlike probabilistic distributions,

all are available at the same time.

Quantum Memory

A quantum memory with n quantum bits is a complex combination of strings of n bits in a Hilbert space. E.g. for $n = 3$:

$$
\begin{array}{rrl}
 & -\frac{1}{2} & \cdot\, 0\,\boxed{0\,0}\,0 \\
+ & \frac{1}{2} & \cdot\, 0\,\boxed{0\,1} \\
+ & \frac{i}{2} & \cdot\, 1\,\boxed{1\,0} \\
- & \frac{i}{2} & \cdot\, 1\,\boxed{1\,1}\,1 \\
 & & \qquad\quad \text{qubit 2}
\end{array}
$$

with a norm condition.

Unlike probabilistic distributions,

all are available at the same time.

Quantum Memory

A quantum memory with n quantum bits is a complex combination of strings of n bits in a Hilbert space. E.g. for $n = 3$:

$$\begin{array}{rrl} & -\frac{1}{2} & \cdot\, 0\,0\,\boxed{0} \\ + & \frac{1}{2} & \cdot\, 0\,0\,\boxed{1} \\ + & \frac{i}{2} & \cdot\, 1\,1\,\boxed{0} \\ - & \frac{i}{2} & \cdot\, 1\,1\,\boxed{1} \end{array}$$

qubit 3

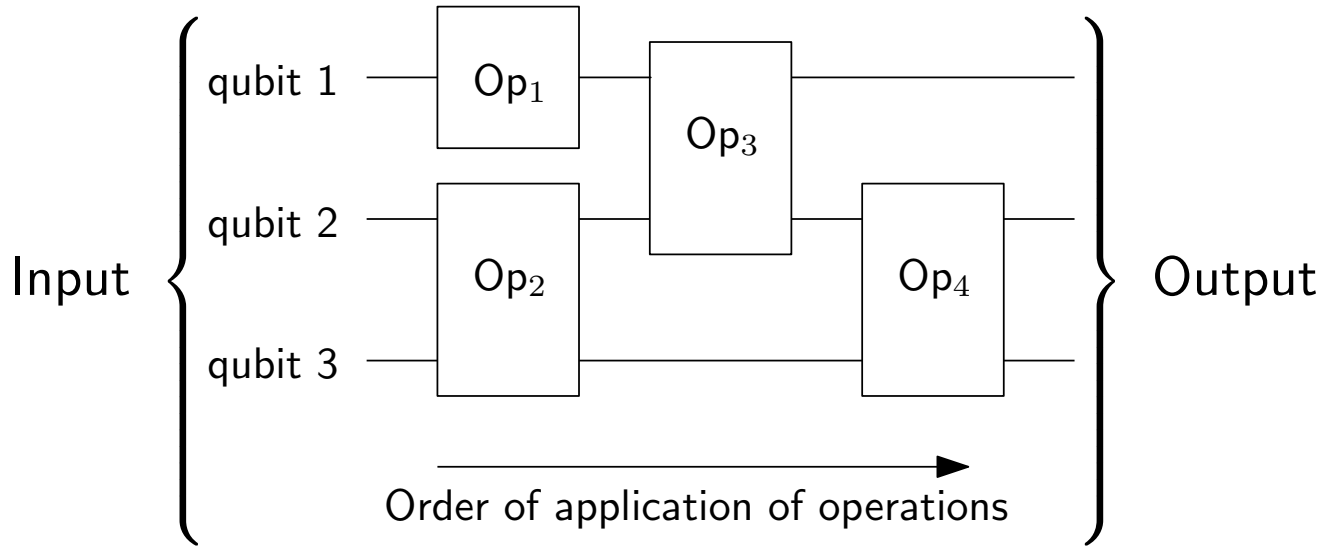
with a norm condition.

Unlike probabilistic distributions,

all are available at the same time.

Quantum Circuit Model

Stream of instructions: a series of elementary gates applied on the quantum memory, that are described by a quantum circuit.

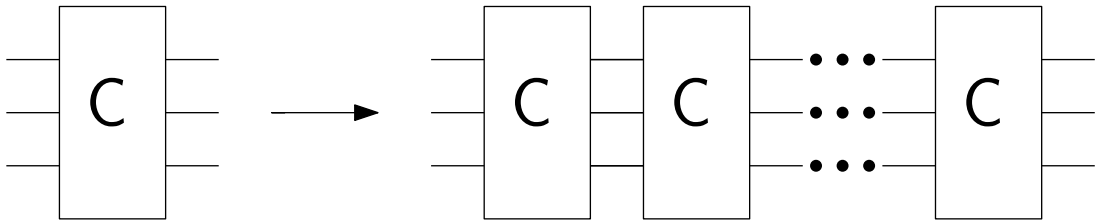


- Each operation is reversible, unitary on the space of states
- Each wire is a quantum bit $\equiv$ a quantum register
- No "quantum loop", "conditional escape" or "branching point".

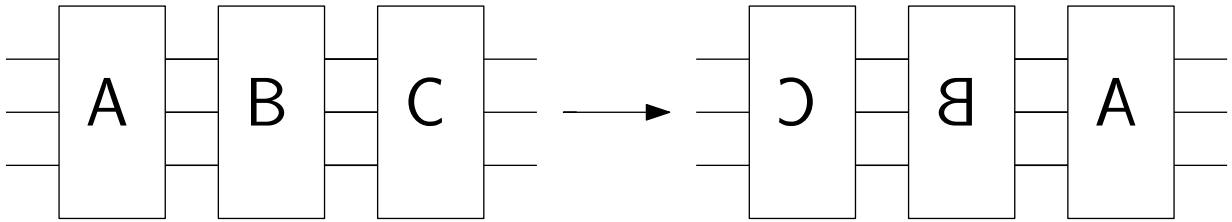
Quantum Circuit Model

Circuit combinators: used to build complex circuits

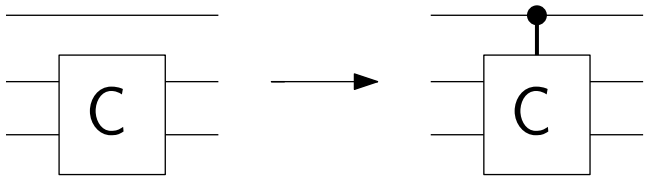
- Repetition



- Inversion

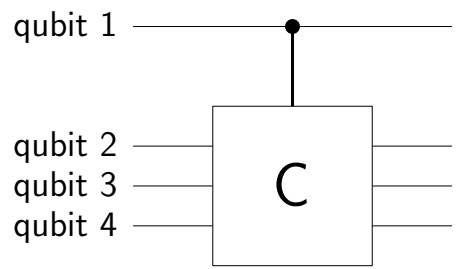


- Control



Quantum Circuit Model

Controlled circuit : conditional action of a circuit



C is applied on qubits 2-4 only when qubit 1 is true:

Suppose that C flips its input bits. Then the above circuit does

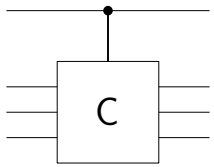
$$\begin{array}{rl} \text{qubit} & 1\,234 \\ \frac{1}{\sqrt{2}} & 1\,010 \\ +\,\frac{1}{\sqrt{2}} & 0\,110 \end{array} \quad \longmapsto \quad \begin{array}{rl} \text{qubit} & 1\,234 \\ \frac{1}{\sqrt{2}} & 1\,101 \\ +\,\frac{1}{\sqrt{2}} & 0\,110 \end{array}$$

This acts as a "quantum test" to be formalized

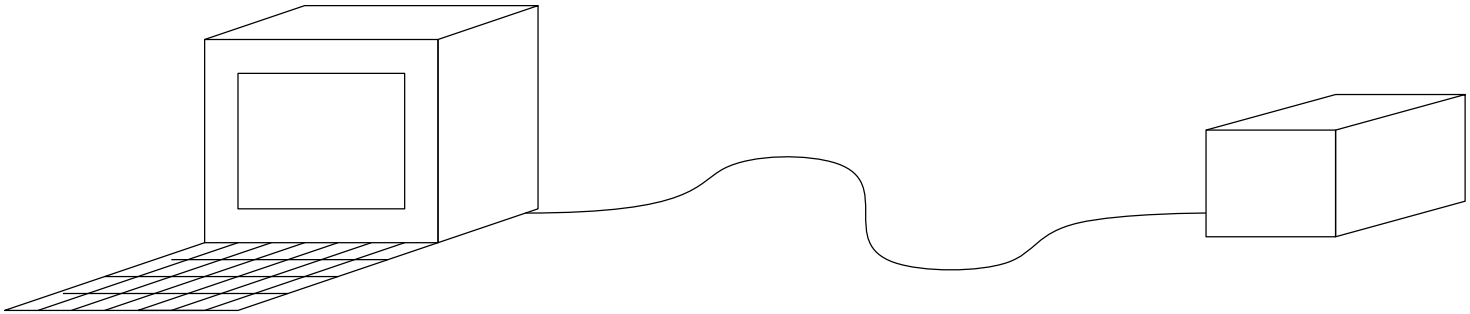
Notion of Control Flow

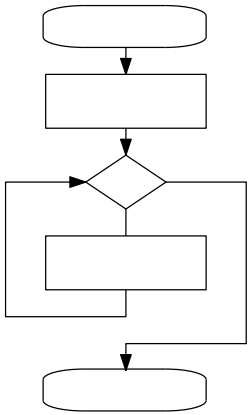
Control flow in quantum computation

has two meanings

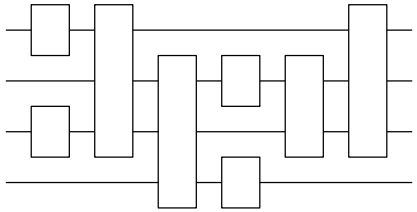
- The control of a gate 

- The (classical) control-flow of the program:





Generates



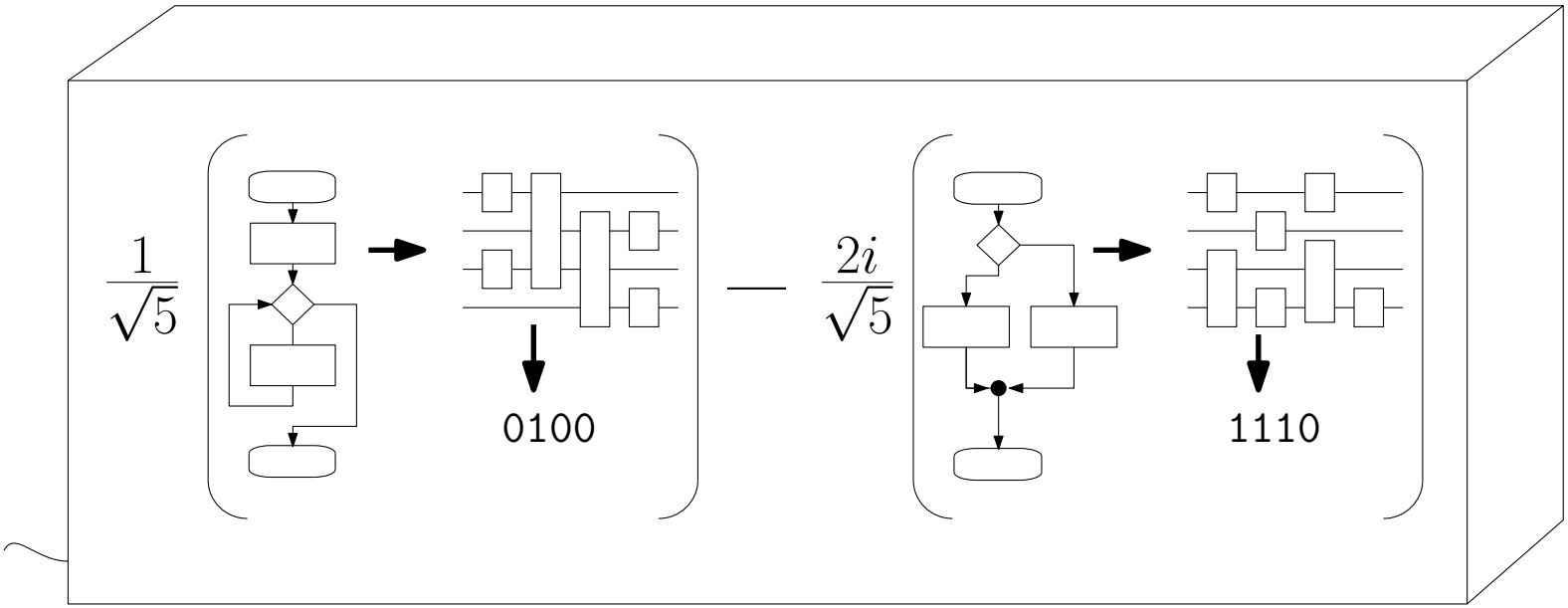
Acts on

$\frac{i}{\sqrt{2}}0010 - \frac{1}{\sqrt{2}}1101$

One circuit as a classical trace of execution

Quantum Control

Exhibiting the "control flow" happening in the quantum memory



Moving control inside the quantum memory:

- Distinct behavior on each distinct state in superposition
- Notion of "quantum" tests
- Notion of "quantum" loops/fixpoints

Quantum Control

Long-term objective

- Understand the structure of quantum operations
- Syntax for building circuit combinators

Challenges

- Preservation of unitarity: norm, orthogonality of states.

$$x : \mathsf{qubit} \vdash \mathtt{if}^\circ\ x\ \mathtt{then}\ x\ \mathtt{else}\ (\mathtt{not}\ x) : \mathsf{qubit}$$

  sends

$$\frac{1}{\sqrt{2}}\mathtt{true} + \frac{1}{\sqrt{2}}\mathtt{false} \quad \longmapsto \quad \frac{2}{\sqrt{2}}\mathtt{true}$$

- No "quantum termination":
  unlike probabilistic behavior, all branches are available

Measure of success for "capturing" quantum control

- Compilation to circuits?
- Modeled as unitary in some Hilbert space?

Roadmap

- Quantum and Control
  - Model of Computation
  - Notion of Control in Quantum Computation
- Previous Approaches
- Pattern Matching and Quantum Control

Previous Approaches

	QML	Quantum λ-calc	Linear-alg. λ-calc	Ying's quantum loops	Circuit desc. lang.	Our proposal
Concrete syntax	✓	✓	✓	✗	✓	✓
Recursive types	✗	✗	✓	✗	✓	✓
Higher-order	✗	✓	✓	✗	✓	✓
Orthogonality and norm	✓	✓	✗	✓	✗	✓
Quantum tests	✓	✗	✓	✓	✗	✓
Quantum loops	✗	✗	✓	✓	✗	✓

[Altenkirch&Grattage] [vanTonder] [Arrighi&Dowek]
[DíazCaro] [Ying] [Quipper,QWIRE]

Limits of Current Approaches

Quantum control flow

- Quantum tests are fine: can be done with regular control
- Quantum loops are the main road block

The following seem slightly incompatible

- Expressive language : linear alg. λ-calc
- Satisfactory loops : Ying's approach
- Preservation of unitarity : QML

Our proposal reconciliates these features

Roadmap

- Quantum and Control
  - Model of Computation
  - Notion of Control in Quantum Computation
- Previous Approaches
- Pattern Matching and Quantum Control

Reversible Pattern Matching

```
f : Nat + Nat -> ...
f (Left 0)    -> ...
f (Left n+1)  -> ...
f (Right n)   -> ...
```

```
g : (Bool x Nat) -> ...
g (True, 0)    -> ...
g (False, n)   -> ...
g (True, n+1)  -> ...
```

Reversible Pattern Matching

```
f : Nat + Nat -> ...
f (Left 0)    -> ...
f (Left n+1)  -> ...
f (Right n)   -> ...
```

```
g : (Bool x Nat) -> ...
g (True, 0)    -> ...
g (False, n)   -> ...
g (True, n+1)  -> ...
```

```
h : Nat + Nat <-> (Bool x Nat)
h (Left 0)    <-> (True, 0)
h (Left n+1)  <-> (False, n)
h (Right n)   <-> (True, n+1)
```

Conditions for reversible maps: isos [James&Sabry,2014]

- Exhaustivity
- Non-overlapping

Test with Reversible Pattern Matching

Consider the isos $U : a \leftrightarrow c$ and $V : b \leftrightarrow d$.

Then

$$\left\{ \begin{array}{lcl} \texttt{Left}(x) & \leftrightarrow & \texttt{Left}(U\,x) \\ \texttt{Right}(y) & \leftrightarrow & \texttt{Right}(V\,y) \end{array} \right\} : \begin{array}{c} a \\ \oplus \\ b \end{array} \leftrightarrow \begin{array}{c} c \\ \oplus \\ d \end{array}$$

- Apply U on values of type a and V on values of type b.

Test with Reversible Pattern Matching

Consider the isos $U : a \leftrightarrow c$ and $V : b \leftrightarrow d$.

Then

$$\left\{ \begin{array}{lcl} \texttt{Left}(x) & \leftrightarrow & \texttt{do}\ \ x \text{ —} \boxed{U} \text{—} x'\ \ \texttt{return}\ \texttt{Left}(x') \\ \texttt{Right}(y) & \leftrightarrow & \texttt{do}\ \ y \text{ —} \boxed{V} \text{—} y'\ \ \texttt{return}\ \texttt{Right}(y') \end{array} \right\} : \begin{array}{c} a \\ \oplus \\ b \end{array} \leftrightarrow \begin{array}{c} c \\ \oplus \\ d \end{array}$$

- Apply U on values of type a and V on values of type b.
- Generalized notion of controlled operation: Extends QML

Test with Reversible Pattern Matching

Consider the isos $U : a \leftrightarrow c$ and $V : b \leftrightarrow d$.

Then

$$\left\{\begin{array}{lcl} \texttt{Left}(x) & \leftrightarrow & \texttt{do}\ \ x \text{—}\boxed{U}\text{—}\ x'\ \ \texttt{return}\ \texttt{Left}(x') \\ \texttt{Right}(y) & \leftrightarrow & \texttt{do}\ \ y \text{—}\boxed{V}\text{—}\ y'\ \ \texttt{return}\ \texttt{Right}(y') \end{array}\right\} : \begin{array}{c} a \\ \oplus \\ b \end{array} \leftrightarrow \begin{array}{c} c \\ \oplus \\ d \end{array}$$

- Apply U on values of type a and V on values of type b.
- Generalized notion of controlled operation: Extends QML
- Inverting done compositionally and syntactically:

$$\left\{\begin{array}{lcl} \texttt{Left}(x') & \leftrightarrow & \texttt{do}\ \ x' \text{—}\boxed{U^{-1}}\text{—}\ x\ \ \texttt{return}\ \texttt{Left}(x) \\ \texttt{Right}(y') & \leftrightarrow & \texttt{do}\ \ y' \text{—}\boxed{V^{-1}}\text{—}\ y\ \ \texttt{return}\ \texttt{Right}(y) \end{array}\right\} : \begin{array}{c} b \\ \oplus \\ a \end{array} \leftrightarrow \begin{array}{c} d \\ \oplus \\ c \end{array}$$

A Language for Isos

Reversible pattern matching: a syntax for circuits...

- with type constructors $\oplus$ and $\otimes$
- generalized notion of control

Following circuit-description languages Quipper/QWIRE we add

- recursive types, e.g. $[a] \equiv 1 \oplus (a \otimes [a])$
- higher-order on isos :
  - iso-variables
  - boxes in circuits can be iso-variables
  - lambda-abstractions $\lambda f.\{\cdots\}$ and application
  - fixpoints : $\mu f.\{\cdots\}$
  - operational semantics: substitution and unfolding

Termination

- Fixpoints are required to terminate on all inputs

An iso describes a bijective map on the sets of values

A Language for Isos

Example of "complex" program: the map operation

Let $f : a \leftrightarrow b$. Define `map` $f : [a] \leftrightarrow [b]$ as

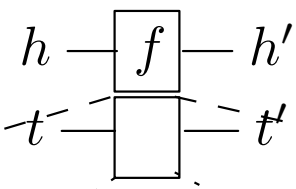
$$\mu \mathbf{g}^{[a]\leftrightarrow[b]}. \left\{ \begin{array}{lcl} [] & \leftrightarrow & [] \\ h : t & \leftrightarrow & \texttt{do} \begin{array}{l} h \text{—} \boxed{f} \text{—} h' \\ t \text{—} \boxed{\mathbf{g}} \text{—} t' \end{array} \texttt{return}\ h' : t' \end{array} \right\}$$

A Language for Isos

Example of "complex" program: the map operation

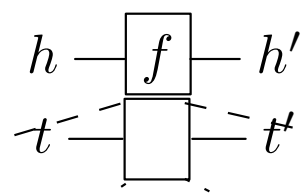
Let $f : a \leftrightarrow b$. Define `map` $f : [a] \leftrightarrow [b]$ as

$$\left\{ \begin{array}{l} [] \quad \leftrightarrow [] \\ h : t \leftrightarrow \texttt{do} \end{array} \right.$$



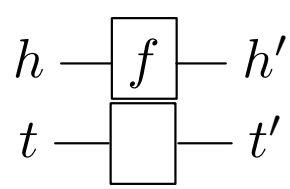
$$\left. \texttt{return}\ h' : t' \right\}$$

$$\left\{ \begin{array}{l} [] \quad \leftrightarrow [] \\ h : t \leftrightarrow \texttt{do} \end{array} \right.$$



$$\left. \texttt{return}\ h' : t' \right\}$$

$$\left\{ \begin{array}{l} [] \quad \leftrightarrow [] \\ h : t \leftrightarrow \texttt{do} \end{array} \right.$$



$$\left. \texttt{return}\ h' : t' \right\}$$

Isos as Unitary Maps (Finite Case)

From classical reversible to quantum:

- linear combination of data $\equiv$ a side effect
- Intuition: the adjunction $\mathrm{FinSet} \rightleftarrows \mathrm{FinVec}$
- Following what is done in the linear algebraic lambda-calculi:

$$a \oplus b \quad \equiv \quad \left\{ \alpha \cdot \mathtt{Left}(u) + \beta \cdot \mathtt{Right}(v) \;\middle|\; u : a, v : b \right\}$$

- $\{\mathtt{Left}(u) \mid u : a\}$ and $\{\mathtt{Right}(v) \mid v : b\}$ are orthogonal subspaces
- In the finite case:
  - Finite dimensional spaces
  - An iso $U : a \leftrightarrow b$ yields a unitary map $a \to b$
  - Superposition of clauses
  - Preservation of orthogonality and norm

Isos as Unitary Maps in $\ell^2(a)$

What about recursive types? Consider $[a] \equiv 1 \oplus (a \otimes [a])$.

- Unrolling the definition:

$$[a] \equiv 1 \oplus a \oplus (a \otimes a) \oplus (a \otimes a \otimes a) \oplus \cdots$$

- $[a]$ infinite dimensional...

A candidate model: The Hilbert space $\ell^2([a])$

- Space of absolute converging sequences indexed with $[a]$.
- Unitary maps U are
  - Surjective
  - Preserving the inner product
  - Bounded: $||Ux|| \leq \rho||x||$ for some $\rho > 0$.

With enough conditions on fixpoints (in this paper: iterators)

- General isos $[a] \leftrightarrow [b]$ yield unitaries $\ell^2([a]) \rightarrow \ell^2([b])$

Isos as Unitary Maps in $\ell^2(a)$

For example

Consider `map Had` of type $[\texttt{Bool}] \leftrightarrow [\texttt{Bool}]$ defined as

$$\mu \mathbf{g}^{[\texttt{Bool}] \leftrightarrow [\texttt{Bool}]}. \left\{ \begin{array}{lcl} [] & \leftrightarrow & [] \\ h : t & \leftrightarrow & \texttt{do} \begin{array}{l} h \text{ — } \boxed{\texttt{Had}} \text{ — } h' \\ t \text{ — } \boxed{\mathbf{g}} \text{ — } t' \end{array} \texttt{return } h' : t' \end{array} \right\}$$

with

$$\texttt{Had} = \left\{ \begin{array}{lcl} \texttt{true} & \leftrightarrow & \frac{1}{\sqrt{2}} \cdot \texttt{true} + \frac{1}{\sqrt{2}} \cdot \texttt{false} \\ \texttt{false} & \leftrightarrow & \frac{1}{\sqrt{2}} \cdot \texttt{true} - \frac{1}{\sqrt{2}} \cdot \texttt{false} \end{array} \right\}$$

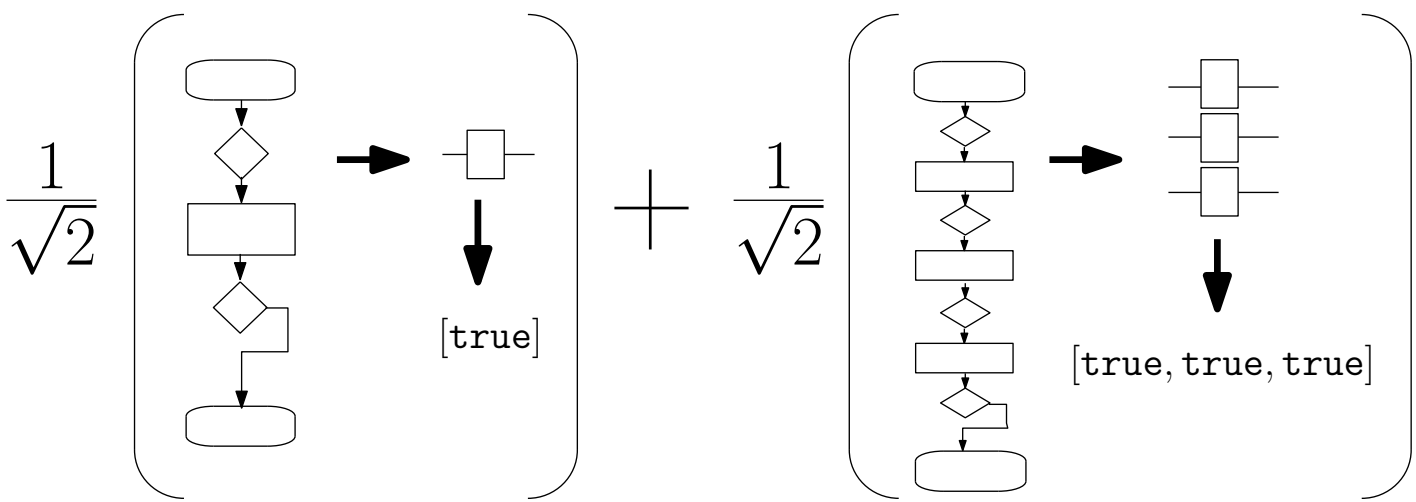
Isos as Unitary Maps in $\ell^2(a)$

For example

Apply `map Had` of type $[\texttt{Bool}] \leftrightarrow [\texttt{Bool}]$ on

$$\frac{1}{\sqrt{2}}[\texttt{true}] + \frac{1}{\sqrt{2}}[\texttt{true}, \texttt{true}, \texttt{true}]$$

and get



A syntatic notion of quantum loop with "quantum termination"?

Concluding

In this paper

- We reconciliate QML, the linear algebraic lambda-calculus and existing approaches to quantum loop.
  - Quantum tests are captured with pattern-matching
  - Quantum loops are simply iterators
- We provide a generalized circuit-construction language with quantum control